HealthNexus Anomaly Detection: ML Report

How the anomaly model was researched, built into the app, measured and tuned, and why each decision was made.

Last updated: 8 October 2026 · model version `personal-baseline-v1`

Summary

HealthNexus flags health readings that look unusual so that a Primary Health Centre (PHC) worker can follow up. Before this work, the app used a placeholder that flagged any value outside a published normal range. We replaced it with a model that starts from published ranges and gradually learns **each person's own normal**, so that people who are healthy but different (for example, fit people with a resting heart rate in the 50s) are no longer flagged every day.

Measure (real wearable data, 71 people, 4,432 readings)	Old placeholder	New model
Normal days wrongly flagged	24.6%	**1.5%**
People flagged on more than half of their days	15 of 71	**0**
Moderate illness-like episodes caught (people with history)	16%	**68%**
Strong illness-like episodes caught	26%	**98%**
Demo catchment: healthy-citizen flags (290 readings)	34	**1**

The model is live in the backend (`backend/app/services/scoring.py`), stores how it judged every reading, and the frontend now shows citizens and health workers **why** a reading was or was not flagged, and how far each person's baseline has come. 13 new backend tests cover it (the full suite has 120), and the evaluation runs the exact app code, not a copy.

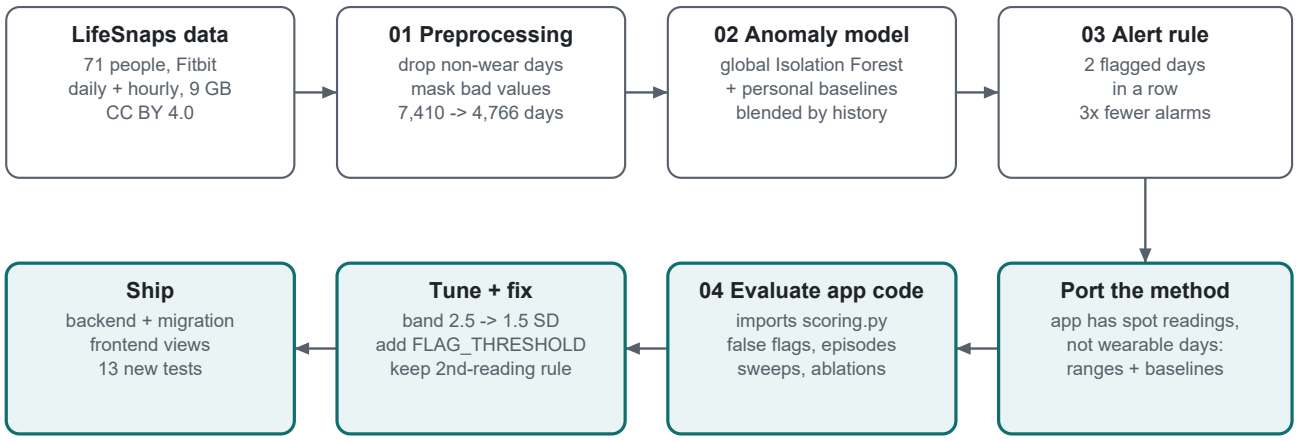
Contents

- 1. Workflow: the research pipeline, the runtime flow in the app, and how one vital is judged
- 2. Metrics: research results, the app model's results, tuning, and checks
- 3. Challenges we hit and how we solved them
- 4. Decisions and why we made them
- 5. Limitations and next steps, and an appendix of files and settings

1. Workflow

The work happened in two loops. A **research loop** on real wearable data found a method that works. An **engineering loop** moved that method into the app, measured the real app code on the same data, and tuned it.

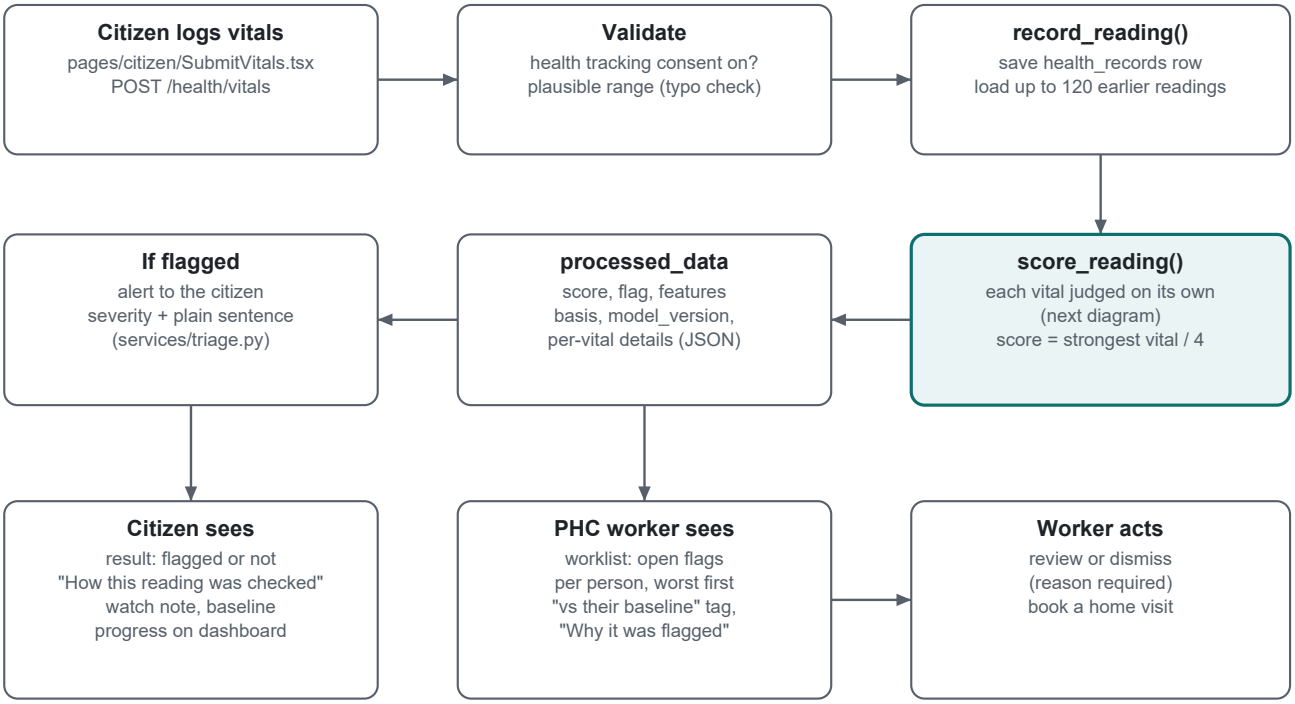
1.1 Research and build pipeline



Step	Where	What happened
Data	`ml/data/rais_anonymized/`	LifeSnaps (Yfantidou et al., 2022): anonymised Fitbit data of 71 people over about 4 months. Open licence; cited in the notebooks.
01 Preprocessing	`ml/notebooks/01_preprocessing.ipynb`	Removed 2,644 days when the watch was not worn (1,440 sedentary minutes, no signal). Set impossible sensor values to empty. Counted each person's history per signal, using past days only.
02 Anomaly model	`ml/notebooks/02_anomaly_model.ipynb`	Global Isolation Forest plus personal rolling baselines, blended by history. Tested by injecting illness-like changes, with a user-level train/test split.
03 Alert rule	`ml/notebooks/03_alert_rule.ipynb`	Alerting only after 2 flagged days in a row cut notifications about 3x.
Port	`backend/app/services/scoring.py`	The app has typed spot readings, not wearable days, so the method was rebuilt for app vitals (section 4 explains why).
04 Evaluate	`ml/notebooks/04_app_scorer.ipynb`	Imports the backend scorer itself and runs it on LifeSnaps heart rate and breathing rate. Threshold sweep, tuning grid, spike test, ablations.
Ship	backend + frontend	Migration 0010, API fields, two new screens, tests. Details in the appendix.

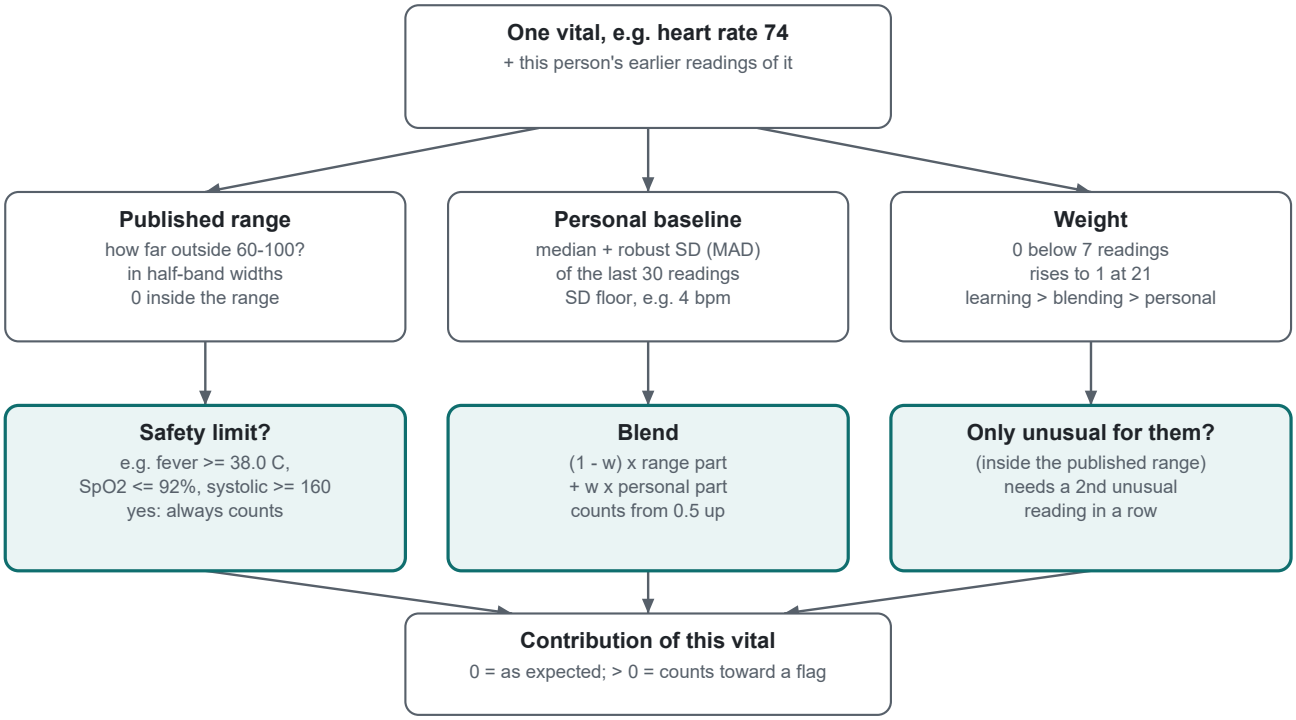
1.2 What happens when a reading is logged

This is the runtime flow inside the app. Everything before `score_reading()` already existed; the model, the stored details and both screens are new.



1.3 How one vital is judged

Every vital in a reading is judged on its own. The reading's score is the strongest vital's contribution divided by 4 (capped at 1), and the existing severity bands (critical, high, moderate, low) are unchanged.



Example. A fit person's usual heart rate is 55. Logging 56 is inside their band, so it is not flagged (the old placeholder flagged it as "below normal"). Logging 75 is inside the 60-100 range but far above their usual, so

the first time it is only a *watch* note; if the next reading is also high, it is flagged. Logging 125 is past the safety limit, so it is flagged at once, whatever their history.

2. Metrics

There are no labels saying "this day was abnormal" in any public wearable dataset we could use, so we measure two things: **false flags** on real normal days, and **detection** of illness-like changes that we inject into real days (3 days in a row of raised heart rate and breathing rate). This is the standard approach when labels do not exist.

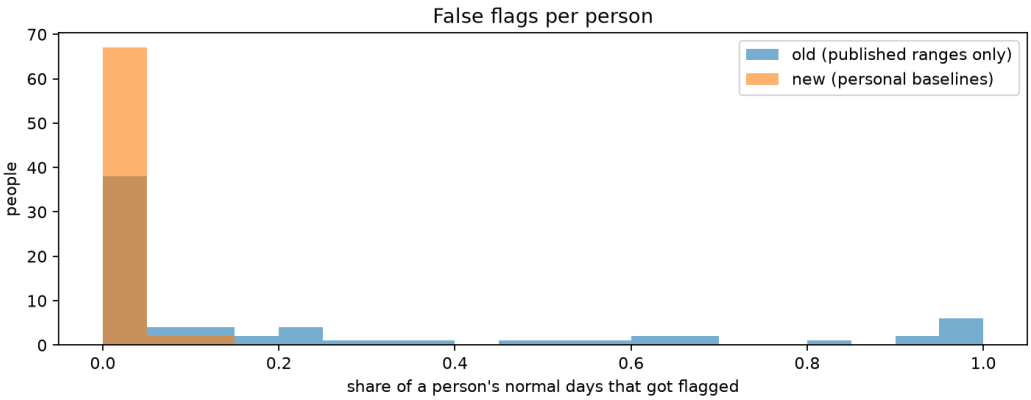
2.1 Research results (wearable model, notebooks 02 and 03)

Injected change (single day)	Global model AUC	Personal baseline AUC	Blend AUC
mild	0.52	0.65	0.67
moderate	0.57	0.83	0.84
strong	0.61	0.90	0.90

- People differ about 3x more from each other than from their own day-to-day changes (resting HR, HRV, breathing rate), so a global model mostly learns "who is this person". This is the main argument for personal baselines.
- For users with a low resting HR, the global model caught 0% of moderate changes and the personal baseline 60%.
- Detection grew with history: 0% (global only) to 22% (blending) to 58% (fully personal).
- Alert rule (notebook 03): notifications fell from 1.07 to 0.38 per user per month. A 3-day rule would give 0.13, but it delays alerts and misses more.

2.2 The app model on real data (notebook 04)

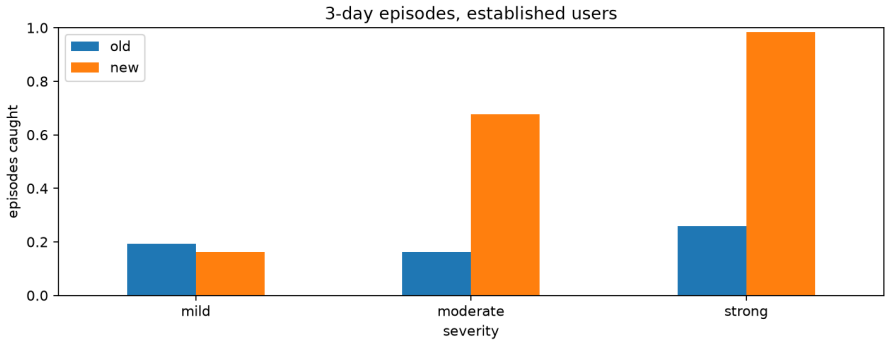
Stage of the person's baseline	Old placeholder false flags	New model false flags
learning (fewer than 7 readings)	23.9%	3.5%
blending (7 to 20 readings)	24.5%	0.8%
personal (21 or more)	24.8%	1.4%
all readings	**24.6%**	**1.5%**



Each bar counts people by the share of their normal days that got flagged. The old scorer flagged 15 people on more than half of their days; nearly all of them have a healthy resting heart rate in the 50s, which is "below normal" by the published range.

3-day episode	When	Old caught	New caught
mild (+6 bpm, +1.5 breaths)	established user	19%	16%
moderate (+10, +2.5)	established user	16%	68%
strong (+15, +4)	established user	26%	98%
moderate	new user (4th reading)	18%	57%
strong	new user	29%	88%

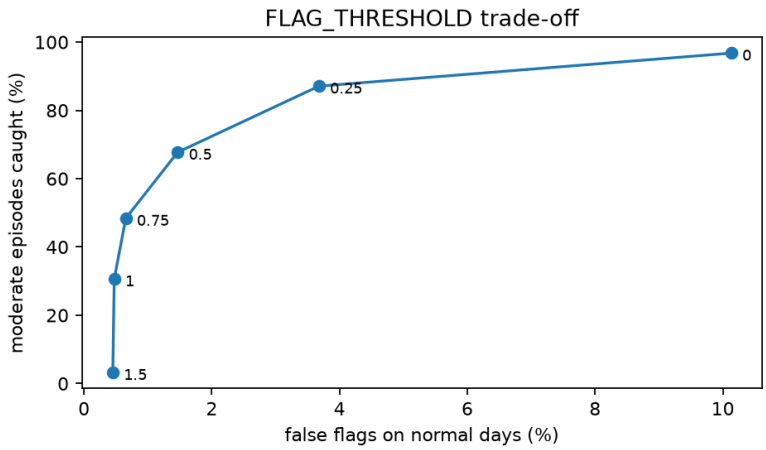
The old scorer's "detection" is mostly chance: it already flags many of these people on normal days, so some episodes land on a flagged day. Most new-model detections happen on day 2 of the episode, which is the second-reading rule at work.



2.3 Tuning

FLAG_THRESHOLD (with the shipped personal band of 1.5 SD). 0.5 is the knee of the curve: below it false flags climb fast, above it detection falls away.

FLAG_THRESHOLD	False flags	Moderate episodes caught
0	10.1%	97%
0.25	3.7%	87%
0.5 (shipped)	1.5%	68%
0.75	0.7%	48%
1	0.5%	31%
1.5	0.4%	3%



Personal band x threshold. The first version (2.5 SD) missed most moderate episodes. 1.5 SD with a 0.5 threshold catches 68% for 1.5% false flags; 1.5 with 0.25 catches more but more than doubles false flags.

PERSONAL_OK_Z	FLAG_THRESHOLD	False flags	Mild	Moderate	Strong
1.5	0.25	3.7%	37%	87%	100%
1.5	0.5	1.5%	16%	68%	98%
2	0.25	3.1%	8%	60%	98%
2	0.5	1.0%	3%	45%	97%
2.5	0.25	2.7%	2%	35%	94%
2.5	0.5	0.9%	2%	16%	76%

2.4 Ablations: does each piece earn its place?

Variant	False flags	Moderate caught	Mild caught
app model (as shipped)	1.5%	68%	16%
no second-reading rule	1.7%	74%	21%
personal band 2.5 SD (first version)	0.9%	16%	2%
full baseline at 14 readings	1.4%	68%	16%
full baseline at 30 readings	1.5%	66%	16%
no spread floor	9.1%	98%	79%
old placeholder	24.6%	16%	19%

Second-reading rule	False flags (clean)	False flags (5% one-off spikes)	Moderate caught
with second-reading rule (shipped)	1.5%	1.5%	68%
without it	1.7%	5.1%	74%

LifeSnaps resting HR is calm and has almost no one-off spikes, so the rule hardly matters on clean data. Real spot readings do have them (measuring right after climbing stairs, for example). With simulated spikes the rule cuts false flags about 3x.

2.5 Checks in the app itself

Check	Result
Demo catchment (27 citizens, 14 days, 388 readings, 7 planned illness episodes)	Healthy citizens: 34 flagged readings with the placeholder, 1 with the model. All 7 episode citizens flagged.
Backend tests	120 pass. 1 onboarding test fails, but it also fails without these changes: the health-centre seed data changed (190 centres, 38 districts) and the test expects 4 districts.
Scorer tests	`tests/test_scoring.py`: 12 unit tests (ramp, safety limits, one-sided SpO2, second-reading rule, weight, robustness, storage). Plus an end-to-end API test in test_modules.py.
Frontend	TypeScript check and production build pass.

3. Challenges and how we solved them

Challenge	What we saw	What we did
The data does not match the app	The research data is daily wearable summaries (resting HR, HRV, sleep). The app gets typed spot readings (BP, SpO2, glucose). Only heart rate and breathing rate overlap.	Kept the method, not the trained Isolation Forest: published ranges play the global model and per-vital personal baselines carry the learning. Evaluated on the two vitals that overlap.
No labels	No dataset says which days were abnormal.	Measured false flags on real normal days, and detection on injected illness-like episodes, at three strengths and two points in a person's history.
Watch not worn	2,329 days had exactly 1,440 sedentary minutes, which means the watch was off.	Dropped non-wear days (2,644 in total) so that "0 steps" was not learned as normal.
Healthy people flagged every day	The placeholder flagged 24.6% of normal days, and 15 people on most days, mostly fit people with a resting HR in the 50s.	Personal baselines: their usual becomes the reference.
Alert fatigue vs safety	With a uniform safety rule, someone with controlled hypertension (usual 145) was still flagged on every reading.	Explicit clinical safety limits per vital (systolic 160, SpO2 92%, temperature 38.0 C...). Below them, people are judged against themselves.
A blending bug, caught by the demo seed	While a baseline blended in, 1 mmHg over the range still flagged; 13 of 20 healthy demo citizens were flagged.	Added FLAG_THRESHOLD (0.5), chosen by a sweep on demo and real data.
A safety limit with the wrong sign	SpO2 was written as "92 or more is dangerous"; normal readings started to flag.	Caught by the API tests; fixed, and a test now checks that SpO2 limits are one-sided.
Too strict at first	The 2.5 SD band caught only 16% of moderate episodes.	Tuning grid: 1.5 SD catches 68% for 1.5% false flags.
Calm research data vs noisy real readings	Without a floor on the spread, LifeSnaps looks great but false flags jump 6x; the rule against one-off spikes looked useless on clean data.	Kept device-noise floors; tested the second-reading rule on simulated spikes, where it cuts false flags about 3x.
Two different "usuals" on screen	Dashboard and record showed mean +/- SD; the model used median +/- MAD.	Both pages now show the model's own baseline, so the deviation a worker sees is the one the model used.
Data leakage	A baseline that includes today, or a test user seen in training, inflates results.	Baselines use earlier readings only (tested); research used a user-level split.
Edge case crash	A run of identical values gave a spread of 0 (division by zero).	Spread is never exactly 0.

4. Decisions and why

Decision	Alternatives considered	Why we chose it
Personal baselines on top of published ranges	Published ranges only; one global ML model	People differ about 3x more from each other than from themselves. Ranges alone flagged 24.6% of normal days; personal baselines 1.5%.
Port the method, not the trained model	Use the Isolation Forest in the app	It was trained on wearable signals the app does not collect; feeding it different vitals would be meaningless.
Median and MAD, not mean and SD	Mean and SD	One odd reading barely moves a median, so a single typo or spike cannot shift someone's normal.
Gradual weight from 7 to 21 readings	Switch at a fixed count	No sudden jump in behaviour; detection rose smoothly with history in the research (0, 22, 58%). 14 vs 21 vs 30 made little difference, and 21 is three weeks of daily readings.
Spread floors per vital (e.g. 4 bpm)	No floor	Without a floor, very regular people get a tiny band and are flagged for device noise (false flags 6x).
Clinical safety limits	Trust the baseline fully	A fever or SpO2 of 91% must always reach a health worker, whatever the person's usual.
FLAG_THRESHOLD 0.5	0.25 / 0.75 / none	The knee of the trade-off curve; without it, tiny excesses flagged while blending.
Personal band 1.5 SD	2.0 / 2.5 SD	2.5 missed most moderate episodes; 1.5 catches 68% at 1.5% false flags.
Second unusual reading for in-range changes	Flag at once	About 3x fewer false flags from one-off spikes, for one day of delay. Same reasoning as the research alert rule.
Store basis, model_version and per-vital details	Store the score only	Every flag can be explained and audited later, even after the model changes.
Show the working to users	Show the flag only	Citizens see why (and the watch note); workers see "vs their baseline" and the per-vital breakdown, so they can trust or question a flag.
Evaluate the real backend file	Re-implement it in the notebook	One source of truth: the numbers describe the shipped code.

5. Limitations and next steps

- Only heart rate and breathing rate could be tested on real data. Blood pressure, SpO2, temperature and glucose use the same code, ranges and floors, but are untested on real data.
- The episodes are synthetic, and LifeSnaps has one calm reading per day; real users log irregularly and more noisily.
- Mild changes (+6 bpm) are mostly missed (16%). That is the size of normal day-to-day variation, and we accept it.
- Next: collect labelled outcomes from PHC reviews ("reviewed" vs "dismissed: false alarm") and use them to re-tune with real labels.
- Next: rows scored before migration 0010 have no stored details; re-scoring them is optional.
- Next: outbreak clustering (community level) is still not started.

Appendix: files and settings

File	Change
`backend/app/services/scoring.py`	The model: published ranges, robust personal baselines, weight ramp, safety limits, FLAG_THRESHOLD, second-reading rule, per-vital details.
`backend/alembic/versions/0010_score_details.py`	Adds processed_data.basis, model_version, details (nullable).
`backend/app/models/processed_data.py`	The three new columns.
`backend/app/services/readings.py`	Stores the details; display baselines now use the model's own baseline.
`backend/app/api/routes/vitals.py`	Result returns checks, watch, model_version; new GET /health/baseline.
`backend/app/api/routes/phc.py, api/views.py`	Basis on worklist and queue items; checks on the record timeline.
`backend/app/schemas/health.py, phc.py`	VitalCheck, VitalBaseline, BaselineStatusOut; Baseline is now usual and spread.
`backend/tests/test_scoring.py, test_modules.py, test_rules.py`	12 unit tests, 1 end-to-end test, updated contract test.
`frontend/src/components/app/ModelInsights.tsx`	ScoreBreakdown, BasisTag, BaselineProgress.
`frontend/src/pages/citizen/SubmitVitals.tsx, Dashboard.tsx`	"How this reading was checked", watch note, "How well we know your normal".
`frontend/src/pages/phc/CitizenRecord.tsx, components/app/phc.tsx`	Model baseline column, "Why it was flagged", basis tags.
`ml/notebooks/04_app_scorer.ipynb, ml/reports/`	Evaluation, metrics JSON and figures used in this report.

Setting	Value	Meaning
MIN_BASELINE_READINGS	7	The personal baseline starts to count.
FULL_BASELINE_READINGS	21	Judged against the person alone.
BASELINE_WINDOW	30	Most recent readings used per vital.
PERSONAL_OK_Z / PERSONAL_UNIT_Z	1.5 / 1.5	Personal contribution starts at 1.5 SD; +1 per 1.5 SD more.
FLAG_THRESHOLD	0.5	Minimum contribution that counts (2.25 SD personally).
SAFETY_LIMITS	per vital	Systolic 85/160, diastolic 50/100, HR 45/120, temperature 35.5/38.0, SpO2 92, breathing 9/24, glucose 65/180.
MIN_SPREAD	per vital	Systolic 5, diastolic 4, HR 4, temperature 0.2, SpO2 1, breathing 1.5, glucose 8, weight 0.8.
SCORE_CEILING	4	A contribution of 4 maps to a score of 1.0 (severity bands unchanged).